key attribute is always set using an attribute called `id`. The `primary_key=` declaration sets the name of the column to use in the table. In the following code, we use an attribute called `id` even though the primary key in the database is `isbn`:

```
book = LegacyBook.new
book.id = "0-12345-6789"
book.title = "My Great American Novel"
book.save
# ...
book = LegacyBook.find("0-12345-6789")
puts book.title    # => "My Great American Novel"
p book.attributes  #=> {"isbn" =>"0-12345-6789",
                   #    "title"=>"My Great American Novel"}
```

Just to make life more confusing, the attributes of the model object have the column names `isbn` and `title`—`id` doesn't appear. When you need to set the primary key, use `id`. At all other times, use the actual column name.

Model objects also redefine the Ruby `id` and `hash` methods to reference the model's primary key. This means that model objects with valid IDs may be used as hash keys. It also means that unsaved model objects can't reliably be used as hash keys (because they won't yet have a valid ID).

One final note: Rails considers two model objects as equal (using `==`) if they are instances of the same class and have the same primary key. This means that unsaved model objects may compare as equal even if they have different attribute data. If you find yourself comparing unsaved model objects (which is not a particularly frequent operation), you might need to override the `==` method.

As we'll see, IDs also play an important role in relationships.

Specifying Relationships in Models

Active Record supports three types of relationship between tables: one-to-one, one-to-many, and many-to-many. You indicate these relationships by adding declarations to your models: `has_one`, `has_many`, `belongs_to`, and the wonderfully named `has_and_belongs_to_many`.